

# TPS65219 PMIC EVM

## Python Libraries Reference Guide

A clear, simple guide to every Python library used in the TPS65219 PMIC EVM measurement application — what each library does and exactly why it is needed.

8 Libraries

3 Built-in

5 Third-Party

0 Extras Needed

### Contents

| Library       | Category             | Role in Project                                           |
|---------------|----------------------|-----------------------------------------------------------|
| PyVISA        | Instrument I/O       | Connects to oscilloscope, sends commands, reads waveforms |
| NumPy         | Numerics             | Waveform maths, edge detection, tRAMP & slew calculation  |
| PyQt5         | GUI Framework        | Entire desktop application — windows, buttons, live plot  |
| Matplotlib    | Visualisation        | Live waveform display and PNG images for PDF report       |
| ReportLab     | PDF Generation       | Assembles the complete multi-page measurement report      |
| sys & os      | System / Built-in    | App startup, shutdown, and cross-platform file paths      |
| io & datetime | Utilities / Built-in | In-memory PNG pipeline and report timestamps              |
| time          | Timing / Built-in    | Hardware settling delays and trigger timeout              |

# PyVISA

Instrument Communication · `pip install pyvisa`

PyVISA is the bridge between Python and the oscilloscope. It uses the VISA standard (Virtual Instrument Software Architecture) to communicate over USB, letting the script send commands and read back measurements automatically — just like pressing buttons on the scope front panel.

| Feature               | What it does                                                                         |
|-----------------------|--------------------------------------------------------------------------------------|
| Open instrument       | Finds and connects to the Keysight EDUX1052A over USB in one line.                   |
| Send SCPI commands    | Sends text commands like <code>:TIM:SCAL 0.2</code> to set timebase, trigger, V/div. |
| Query measurements    | Asks the scope for voltage, rise time, frequency and gets a number back.             |
| Read binary waveforms | Fetches 2000 raw byte samples directly from scope memory as a NumPy array.           |
| Auto-scan all ports   | Loops through all connected USB instruments and picks the oscilloscope.              |
| Timeout control       | Sets a time limit so the script does not hang if the scope is busy.                  |

**Used for in this project:** Connecting to the scope, configuring channels & trigger, reading waveform data and built-in measurements.

# NumPy

Numerical Computing · `pip install numpy`

NumPy is the maths engine. When the scope sends 2000 raw byte values, NumPy converts them to real voltages, smooths out noise, detects edges, and measures rise times — all in a fraction of a second using fast array operations.

| Feature                         | What it does                                                                       |
|---------------------------------|------------------------------------------------------------------------------------|
| Array maths                     | Treats 2000-point waveforms as a single object — scale, offset, add in one step.   |
| <code>np.where()</code>         | Finds the exact sample where voltage crosses a threshold (e.g. 10%, 98% of swing). |
| <code>np.diff()</code>          | Computes the slope at every sample to detect rising and falling edges.             |
| Moving average                  | 5-point smoothing (cumsum trick) removes ADC noise from raw scope data.            |
| <code>np.max / min / std</code> | Finds signal swing, noise floor, and stability — used in every measurement.        |
| Type conversion                 | Converts raw uint8 bytes to float64 volts using YINC, YORIG, YREF from scope.      |

**Used for in this project:** Waveform conversion, smoothing, edge detection, tRAMP calculation, and slew-rate measurement.

# PyQt5

Desktop GUI Framework · `pip install PyQt5`

PyQt5 builds the entire visual application — the dark-themed window, rail selection radio buttons, live waveform canvas, event log, and snapshot gallery. It also provides the event loop that keeps the UI responsive while data is streaming from the scope.

| Feature                            | What it does                                                                     |
|------------------------------------|----------------------------------------------------------------------------------|
| <b>QMainWindow / QWidget</b>       | The main window and all panels — waveform, controls, snapshots.                  |
| <b>QTimer (50 ms)</b>              | Fires every 50 ms to fetch and redraw the live waveform without freezing the UI. |
| <b>QRadioButton / QButtonGroup</b> | Rail selection (BUCK1/2/3, LDO1–4) with exclusive single-choice logic.           |
| <b>QListWidget + QIcon</b>         | Snapshot gallery — each captured waveform shown as a thumbnail in a list.        |
| <b>QFileDialog</b>                 | Native OS save dialog so the user can choose where to save the PDF.              |
| <b>QMessageBox</b>                 | Pop-up alerts for connection errors, warnings, and save confirmations.           |
| <b>Signals &amp; Slots</b>         | Connects button clicks and timer events to Python functions automatically.       |

**Used for in this project:** The complete GUI — selection window, live waveform, event log, snapshot list, and PDF save dialog.

## Matplotlib

Plotting & Visualisation · `pip install matplotlib`

Matplotlib draws the live dual-channel waveform inside the PyQt5 window and generates high-resolution PNG images for the PDF report. A special off-screen renderer (Agg) creates images in memory without ever opening a separate window.

| Feature                         | What it does                                                                 |
|---------------------------------|------------------------------------------------------------------------------|
| <b>FigureCanvasQTAgg</b>        | Embeds a Matplotlib figure directly inside the PyQt5 window as a widget.     |
| <b>Dual Y-axis (twinx)</b>      | CH1 (gold) and CH2 (green) each have their own independent voltage axis.     |
| <b>Blit rendering</b>           | Only redraws the two waveform lines each frame — keeps 50 ms updates smooth. |
| <b>FigureCanvasAgg</b>          | Off-screen renderer: generates PNG bytes in RAM for PDF, no popup window.    |
| <b>axhline / axvline / text</b> | Draws threshold markers (10%, 98%, 90% VNOM) and timing lines on captures.   |
| <b>Custom styling</b>           | Dark background, gold/green traces, dashed grid — matches the app theme.     |

**Used for in this project:** Live waveform display in the GUI, and generating waveform PNG images embedded in the PDF report.

## ReportLab

PDF Generation · `pip install reportlab`

ReportLab assembles the final measurement report PDF. It lays out the cover page, configuration tables, per-snapshot pages with waveform images, the complete measurement log, and a specification reference — all with professional formatting and colour-coded PASS/FAIL results.

| Feature                    | What it does                                                                         |
|----------------------------|--------------------------------------------------------------------------------------|
| SimpleDocTemplate          | Handles page size (A4), margins, and automatic page breaks for the whole document.   |
| Platypus (Paragraph/Table) | High-level layout engine — stacks text, tables, and images in the correct order.     |
| TableStyle                 | Colours header rows, alternates row backgrounds, highlights PASS green and FAIL red. |
| ParagraphStyle             | Custom font, size, colour, and alignment for every text style in the document.       |
| RImage                     | Embeds Matplotlib PNG waveform images into the report at precise dimensions.         |
| HRFlowable                 | Draws horizontal divider lines between sections for a clean layout.                  |
| PageBreak                  | Separates cover, summary, snapshot pages, log, and spec reference cleanly.           |

**Used for in this project:** Generating the complete multi-page PDF report with tables, embedded waveforms, and PASS/FAIL results.

## sys & os

Python Standard Library · Built-in, no install needed

These built-in modules handle the application lifecycle and file system. sys manages the Qt event loop and clean shutdown, while os builds file paths that work correctly on Windows, macOS, and Linux without any code changes.

| Feature                 | What it does                                                                 |
|-------------------------|------------------------------------------------------------------------------|
| sys.argv                | Passes arguments to QApplication — required by PyQt5 to start correctly.     |
| sys.exit()              | Shuts down the application cleanly when the window is closed.                |
| os.path.join()          | Builds file paths that work on all operating systems (no hardcoded slashes). |
| os.path.expanduser('~') | Resolves the user home directory for the default PDF save location.          |

**Used for in this project:** Application startup, clean shutdown, and cross-platform PDF save path construction.

## io & datetime

Python Standard Library · Built-in, no install needed

io provides in-memory byte buffers so PNG images pass directly from Matplotlib to ReportLab without writing any temporary files to disk. datetime stamps every PDF report with the exact date and time of generation.

| Feature                 | What it does                                                                   |
|-------------------------|--------------------------------------------------------------------------------|
| io.BytesIO              | In-memory buffer — Matplotlib writes PNG bytes in; ReportLab reads them out.   |
| datetime.datetime.now() | Captures current date and time for the PDF cover page and report filename.     |
| strftime()              | Formats timestamps: 2025-07-18_143022 for filenames, 18 July 2025 for display. |

**Used for in this project:** Zero-disk PNG pipeline from Matplotlib to ReportLab, and timestamping every saved report.

# time

Python Standard Library · Built-in, no install needed

After every configuration command sent to the oscilloscope, the hardware needs a short pause before the new setting takes effect. `time.sleep()` inserts these settling delays, and `time.time()` measures elapsed seconds when waiting for a trigger event.

| Feature                               | What it does                                                                            |
|---------------------------------------|-----------------------------------------------------------------------------------------|
| <code>time.sleep(0.05 – 0.5 s)</code> | Pauses the script after each scope command so hardware can settle before the next read. |
| <code>time.time()</code>              | Records the start time when waiting for a trigger; stops waiting after the timeout.     |

**Used for in this project:** Hardware settling delays after scope configuration commands, and trigger timeout measurement.

# Quick Reference Summary

| Library       | Type        | Install                | Key Use in Code                                             |
|---------------|-------------|------------------------|-------------------------------------------------------------|
| PyVISA        | Third-party | pip install pyvisa     | query_binary_values() — fetches 2000-point scope waveform   |
| NumPy         | Third-party | pip install numpy      | np.where(sig >= threshold) — finds rise/fall crossing point |
| PyQt5         | Third-party | pip install PyQt5      | QTimer(50ms) — live waveform refresh without UI freeze      |
| Matplotlib    | Third-party | pip install matplotlib | FigureCanvasAgg — PNG in memory, no popup window            |
| ReportLab     | Third-party | pip install reportlab  | Platypus Table + RImage — measurement report with waveforms |
| sys & os      | Built-in    | —                      | sys.exit() + os.path.join() — shutdown & file paths         |
| io & datetime | Built-in    | —                      | io.BytesIO — passes PNG bytes from Matplotlib to ReportLab  |
| time          | Built-in    | —                      | time.sleep(0.4) — scope settling after :TIM:SCAL command    |

## Install All Third-Party Libraries at Once

```
pip install pyvisa numpy PyQt5 matplotlib reportlab
```